

Vision-Language Engineering

CLIP, Qwen3-VL, visual document understanding and multimodal RAG

GHANMI Helmi

AI Engineer | Former Researcher in Applied Mathematics

<https://www.researchgate.net/profile/Ghanmi-Helmi>

Reference runtime: Python 3.12

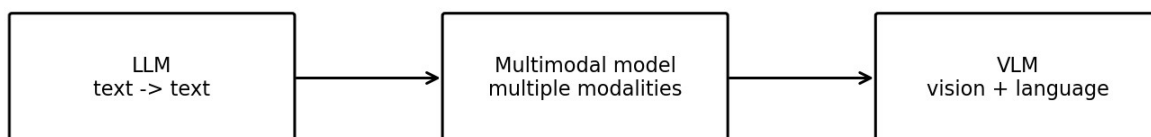
Model/API notes verified 27 Aug 2026

How to use this course

This course starts from the distinction between language models and vision-language models, then connects model usage to a production-shaped repository. The goal is not only to run a model, but to understand how visual evidence enters retrieval and grounded generation.

- Understand LLM vs multimodal model vs VLM vs OCR vs embeddings vs RAG.
- Use CLIP for image-text similarity and zero-shot classification.
- Run Qwen3-VL-2B-Instruct from the Hugging Face Hub/cache or from a local offline snapshot.
- Turn diagrams, screenshots and document pages into structured, traceable RAG chunks.
- Start with text-only retrieval, then progress to Qwen3-VL multimodal embedding and reranking.
- Evaluate extraction, retrieval and final-answer quality before scaling.

1. LLM, multimodal model and VLM



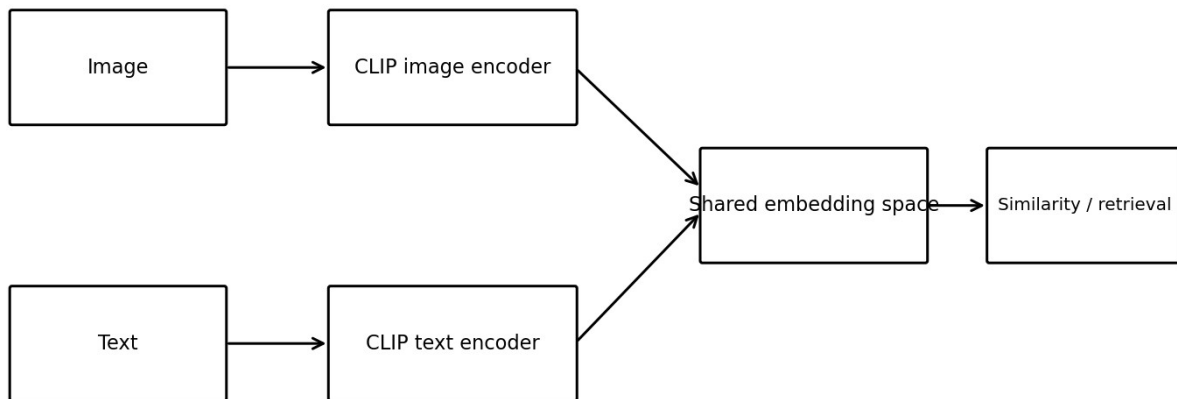
VLM is a multimodal specialization focused on visual-language understanding.

OCR recognizes characters. A VLM can also reason about layout, arrows, groups, tables and visual relationships.

Term	Simple definition	Typical role
LLM	Primarily text tokens -> text tokens	Reasoning/generation after evidence is textualized
Multimodal model	General family that accepts multiple modalities	Text, image, audio, video combinations
VLM	Vision + language specialization	Pages, figures, screenshots, diagrams, VQA
OCR	Character recognition	Recover literal text, not necessarily visual relations
Embedding model	Maps inputs to vectors	Similarity and retrieval
RAG	Retrieve evidence before generation	Ground answers in relevant chunks

Key idea: OCR may read the labels inside a diagram, while a VLM can also explain that an arrow connects Service A to PostgreSQL. For document AI, that visual relationship is often the information that matters.

2. CLIP: the shared embedding-space foundation



Qwen3-VL instead consumes image + instruction and generates an answer, description or structured output.

CLIP learns image and text representations that can be compared in one space. It is excellent for similarity, retrieval and zero-shot classification. It is not a chat-style generative model: CLIP scores relationships; Qwen3-VL generates language conditioned on visual evidence.

```
from PIL import Image
from vlm_engineering.clip import CLIPEncoder

image = Image.open("data/example.jpg").convert("RGB")
clip = CLIPEncoder()
print(clip.zero_shot_classify(image, ["cat", "dog", "airplane"]))
```

3. Qwen3-VL-2B-Instruct

Qwen/Qwen3-VL-2B-Instruct is an image-text-to-text model published under Apache-2.0. The current Hugging Face model page exposes both a high-level image-text-to-text pipeline and direct loading with AutoProcessor + AutoModelForMultimodalLM.

Use case	What the VLM contributes
Image description	Rich factual caption rather than a similarity score
Visual question answering	Answer a question grounded in one or more images
Diagram analysis	Recover components, arrows, groups and spatial relations
Screenshot understanding	Interpret dashboards, UI states and forms
Table/document understanding	Combine visible text with layout/structure
Structured extraction	Return JSON for validation and downstream code

4. Three model-loading modes

4.1 Hub/cache mode

```
from vlm_engineering.qwen import QwenVLModel
```

```
model = QwenVLModel.from_hub("Qwen/Qwen3-VL-2B-Instruct")
print(model.generate("data/example.jpg", "Describe this image precisely."))
```

The first run may download weights. Later runs reuse the Hugging Face cache.

4.2 Explicit local/offline mode

```
vlm-lab download-model \
  --model-id Qwen/Qwen3-VL-2B-Instruct \
  --output models/Qwen3-VL-2B-Instruct

vlm-lab describe data/example.jpg \
  --model-path models/Qwen3-VL-2B-Instruct
```

The local loader sets `local_files_only=True`. This is useful for reproducible deployments, air-gapped environments and explicit model lifecycle management.

4.3 `trust_remote_code`

The project keeps `trust_remote_code=False` by default. Current Qwen3-VL is integrated into Transformers, so normal usage does not need repository-provided Python code. Enable remote custom code only when a model genuinely requires it and you trust the source.

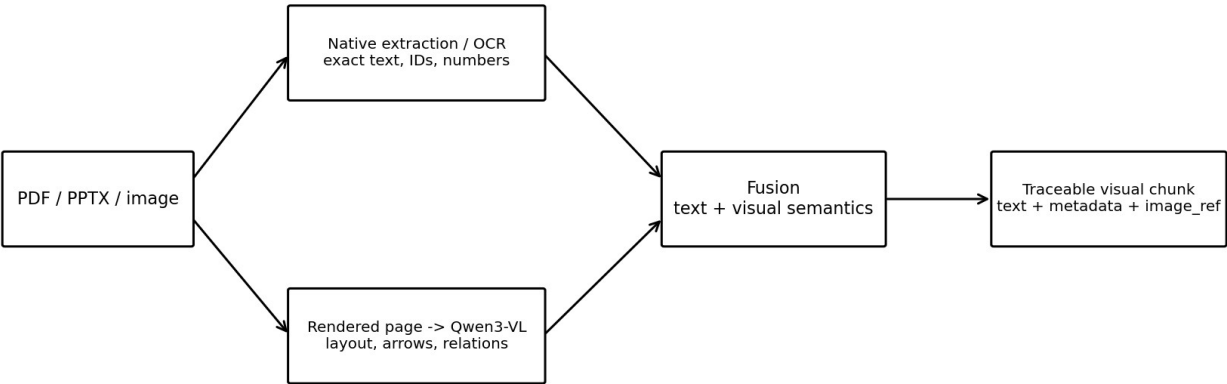
5. Structured visual understanding for RAG

For RAG, "describe the image" is often too vague. Ask for stable fields that can be validated and indexed.

```
{
  "page_type": "diagram|table|text|mixed|photo|screenshot",
  "title": "...",
  "summary": "...",
  "entities": [...],
  "relations": [{"source": "...", "relation": "...", "target": "...", "evidence": "...}],
  "important_text": [...],
  "uncertainties": [...]}
}
```

Keep uncertainties explicit. A useful extraction says "arrow unreadable" rather than inventing a confident relationship.

6. Should a VLM replace OCR?



Usually, do not frame the architecture as VLM versus OCR. Native extraction/OCR is often better for exact identifiers, numbers and long literal text. A VLM is better at visual relationships, layout, groups, diagrams and mixed pages. Fuse both and preserve provenance.

- Use native parsing/OCR first when deterministic literal text is available.
- Render each page/slide so the VLM can see layout and diagrams.
- Call the VLM on every page initially, then add a selector when scaling.
- Store native text and VLM output separately before fusion.
- Keep image_ref so the final VLM can re-check the original visual evidence.

7. Semantic visual chunks

A good chunk is an autonomous unit of meaning, not only a fixed token window. A diagram should usually stay attached to its title, legend, entities, relations, parent context and original image reference.

Field	Purpose
chunk_id	Stable traceability
document_id / page	Return to the source
chunk_type	diagram, table, mixed, screenshot...
text	Fused native + visual retrieval text
entities / relations	Structured semantics
image_ref	Original visual proof
parent_context	Hierarchy when sub-chunking

For narrative text, limited overlap can help. For structured diagrams, prefer parent-child context over arbitrarily cutting relationships in half.

8. Visual RAG with only a text embedder

A VLM can bridge a visual corpus into an existing text-only RAG stack:

image/page -> Qwen3-VL description + JSON -> retrieval text -> text embedder -> vector DB

This pattern is practical when the organization already owns a good text embedder. The important constraint is that retrieval can only find visual facts that the VLM description actually expressed. Keep the original image for final grounding.

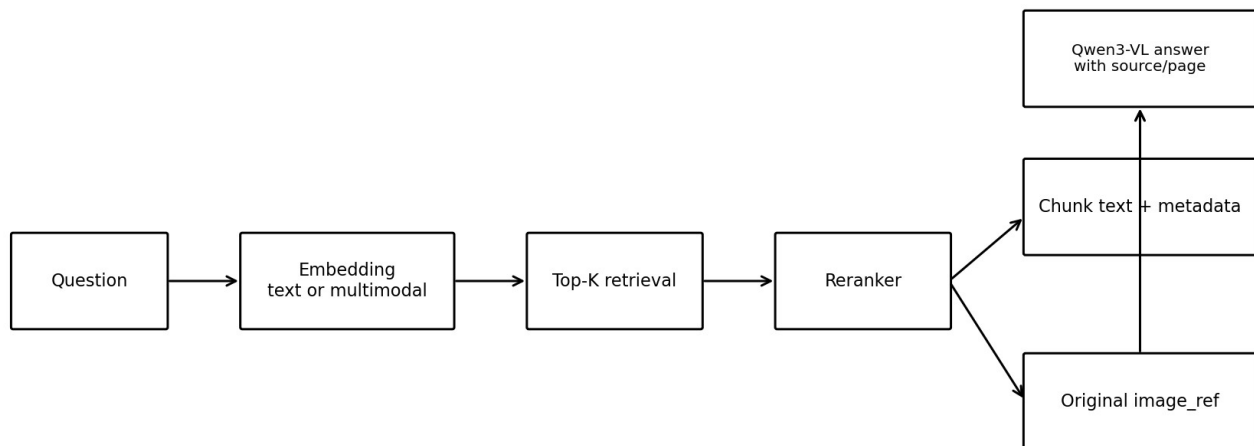
9. True multimodal retrieval

Qwen3-VL-Embedding-2B is designed for multimodal retrieval and can represent text, images, screenshots, video or mixed text+image inputs. Its SentenceTransformers integration provides a direct engineering path. The model card reports up to 2048 embedding dimensions and a 32k context length.

```
from vlm_engineering.retrieval import QwenMultimodalEmbedder

embedder = QwenMultimodalEmbedder()
queries = ["Find the architecture diagram containing a cache and a database."]
docs = ["text note", "data/architecture.png", {"text": "Payment architecture",
"image": "data/architecture.png"}]
q = embedder.encode(queries, prompt="Retrieve relevant technical document evidence.")
d = embedder.encode(docs)
print(q @ d.T)
```

10. Reranking and final grounded answer



Qwen3-VL-Reranker-2B is the precision stage: the embedding model performs broad recall, then the reranker scores query-document pairs more carefully. Retrieve the best chunk text and image_ref, then give both to Qwen3-VL-Instruct for the final answer.

```
from vlm_engineering.retrieval import QwenMultimodalReranker

reranker = QwenMultimodalReranker()
scores = reranker.score("Which diagram shows PostgreSQL?", candidates)
```

11. Production evaluation

Layer	Useful metrics
Visual extraction	valid JSON rate, entity/relation exact match or F1, hallucinated relation rate
Retrieval	Recall@K, MRR, image/page hit rate
Reranking	MRR / nDCG / top-1 relevance
Final answer	grounded correctness, citation/source accuracy, abstention on ambiguity
Operations	latency, GPU memory, throughput, failure rate, model/prompt version

Start with 30-100 representative pages and questions. A controlled evaluation set and traceable schema are often more valuable than immediately switching to a larger model.

12. Project quickstart

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install -e ".[all,dev,notebooks]"
```

```
python -m ruff check src tests examples
python -m mypy src/vlm_engineering
python -m pytest
```

```
vlm-lab describe data/example.jpg
vlm-lab analyze data/page_001.png
```

13. Practical exercises

1. Run CLIP zero-shot classification on one image and inspect label sensitivity.
2. Ask Qwen3-VL the same image question with a vague prompt and a structured prompt; compare downstream usability.
3. Take a technical diagram and compare OCR/native text with VLM entity/relation extraction.
4. Create visual chunks for ten pages and test whether text-only retrieval finds visual facts after adding VLM descriptions.
5. Replace the text embedder with Qwen3-VL-Embedding and compare Recall@K.
6. Add reranking and measure whether the correct page moves higher in the ranked list.

14. References and freshness

- Qwen/Qwen3-VL-2B-Instruct - <https://huggingface.co/Qwen/Qwen3-VL-2B-Instruct>
- Qwen/Qwen3-VL-Embedding-2B - <https://huggingface.co/Qwen/Qwen3-VL-Embedding-2B>
- Qwen/Qwen3-VL-Reranker-2B - <https://huggingface.co/Qwen/Qwen3-VL-Reranker-2B>
- Hugging Face Hub download guide - https://huggingface.co/docs/huggingface_hub/guides/download
- Course basis supplied with this project: VLM, semantic chunking and multimodal RAG (28-page French teaching PDF).

Interfaces and model recommendations evolve. For production, pin model revisions, record prompt versions and re-run evaluations before upgrades.